

AutoCivil-Lab v1

Single-page repo summary based on repository evidence only

What It Is / Who It Is For

A Python-based engineering ML lab for **concrete compressive-strength regression**. It combines dataset preparation, baseline training, governed model research, engineering validation, uncertainty analysis, inverse mix design, and a Flask dashboard over generated artifacts.

Primary user/persona: a civil/materials engineering practitioner or engineering ML researcher working with concrete mix data, model improvement, and mix-design checks.

What It Does

- Ingests a local workbook, downloaded UCI data, or synthetic fallback data and normalizes it to `data/concrete_data.csv`.
- Adds engineered mix and binder features through `feature_engineering.py`.
- Trains a baseline regressor and evaluates candidate models with CV, holdout metrics, and composite scoring.
- Runs a governed scout -> confirm -> keep/revert loop with experiment memory and diversity tracking.
- Uses Ollama, OpenAI, or hybrid proposal backends with deterministic fallback when LLMs are unavailable.
- Applies engineering rule checks and uncertainty calibration before accepting results.
- Produces reports, plots, inverse mix-design suggestions, benchmark artifacts, and a Flask dashboard/API over outputs/.

How It Works

- **Inputs:** `config.yaml` selects data mode; `generate_data.py` loads `concrete_combined.xlsx` or UCI/synthetic data and writes the normalized dataset.
- **Core ML:** `train.py` loads data, applies engineered features, splits train/test, trains models, and saves baseline artifacts; `validator.py` scores engineering plausibility.
- **Research orchestration:** `research_loop.py` reads `research_brief.md`, the current `research_lab.py` surface, model configs, experiment memory, and optional proposals from `proposal_engine.py` via `llm_backend.py`.
- **Governance:** accepted runs sync canonical outputs such as `final_metrics.json`, `best_search_result.json`, `best_search_model.pkl`, plus memory/diversity logs.
- **Post-processing/UI:** `report.py`, `uncertainty.py`, `benchmark.py`, and `design_tool.py` add artifacts; `dashboard.py` serves `/api/*`, `/outputs/*`, `/health`, and `/`.

Data flow: `workbook/UCI source` -> `generate_data.py` -> `train.py` / `research_loop.py` -> `outputs/*.json|csv|pkl|png` -> `dashboard.py`.

How To Run

1. Install dependencies: `pip install -r requirements.txt`.
2. Build the normalized dataset using the current workbook-configured setup: `python generate_data.py`.
3. Train the baseline artifacts: `python train.py`.
4. Run the governed research loop and regenerate reports: `python research_loop.py --with-report`.
5. Start the dashboard: `python dashboard.py` and open `http://localhost:5050`.

Optional repo-backed settings: set `OPENAI_API_KEY` if using the OpenAI proposal backend, and `DASHBOARD_PASSWORD` if you want dashboard auth.